PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 2
3. Product Purpose 3
4. System Functionalities 4-5
5. User Stories & Use Cases 6
6. Features Included (Scope Definition) 7
7. Features Not Included (Scope Control) 7
8. Assumptions & Constraints 8
9. Risks & Questions Throughout Development 8

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: RefundFlow AI

Version: 1.0

1. Project Overview

- **Problem Statement:** Consumers often buy from many online platforms, but refund procedures differ by platform, order type, return reason, evidence requirement, and dispute status. Users usually provide messy text such as chat messages, incomplete order details, screenshots, or emotional complaints. This causes repeated backand-forth communication, missed evidence, wrong refund channel selection, and delayed refund outcomes.
- **Target Domain:** Customer request handling, refund case preparation, administrative workflow automation, and platform specific task orchestration.
- **Proposed Solution Summary:** RefundFlow AI is a GLM-powered workflow engine that converts unstructured refund requests into structured refund cases. It detects the platform and refund reason, checks missing information, generates platform-specific workflow tasks, prepares a refund claim package, and tracks the case status until the next action is clear.

© UMHackathon 2026

Email: umhackathon@um.edu.my

2. Background & Business Objective

- **Background of the Problem:** Refund processes are fragmented as every buying platform has its own support channel, evidence requirements, return windows, and dispute wording. A user may buy from Shopee, Lazada, TikTok Shop, Amazon, or another marketplace, but the user usually does not know what details are required before contacting support. As a result, refund cases are often delayed because the first submission is incomplete. For support teams, manually reading refund messages is repetitive. Agents need to identify the platform, product, order ID, refund reason, evidence, and next step before they can act. These decisions are exactly the kind of multi-step workflow that can be improved by an AI reasoning layer.

  Importance of Solving This Issue:

- Reduces the time required to prepare a complete refund request.
- Helps users understand what information is missing before submission.
- Improves support consistency by converting messy text into structured cases.
- Reduces failed or rejected refund submissions caused by incomplete evidence.
- Shows practical use of GLM as a reasoning and orchestration layer instead of a simple chatbot

- **Strategic Fit / Impact:** The project fits the AI Systems & Agentic Workflow Automation domain because the system does not only reply to the user. It decides the workflow stage, detects missing data, generates tasks, selects the next action, and produces structured output for downstream components. If GLM is removed, the system loses its ability to understand messy refund messages and coordinate the next workflow step.

© UMHackathon 2026
Email: umhackathon@um.edu.my

3. Product Purpose

3.1. Main Goal of the System: The main goal of RefundFlow AI is to guide users through the refund process across different buying platforms by turning unstructured messages into platform-ready refund workflows.

3.2. Intended Users (Target Audience):

User groups	Needs
Online shoppers	Need help preparing refund requests and understanding missing documents.
Customer support agents	Need structured case information instead of messy long messages.
Small business sellers	Need faster triage of return/refund issues
Marketplace operations teams	Need consistent refund categorisation and task tracking.
Hackathon judges / demo users	Need to see a clear AI workflow engine with backend, frontend, GLM, and testing evidence.

© UMHackathon 2026

Email: umhackathon@um.edu.my

4. System Functionalities

4.1. Description: RefundFlow AI operates as a stateful refund workflow engine. Users enter a refund problem in natural language, optionally with structured fields such as platform, order ID, item name, purchase amount, and evidence description. The backend sends the prepared context to the GLM reasoning layer. The response is parsed into structured workflow output containing intent, confidence, missing information, case stage, next action, platform channel, and tasks.

4.2. Key Functionalities:

Functionality	Description
Unstructured Refund Intake	Accepts messy refund descriptions such as complaints, chat-like messages, and incomplete order details.
Refund Reason Classification	Classifies whether the issue is damaged item, missing parcel, wrong item, late delivery, cancellation, or general refund request
Missing Information Detection	Checks whether important details such as platform, order ID, item name, amount, and evidence are missing.
Platform-Specific Workflow Orchestration	Generates task steps according to selected platform adapter and refund reason
Case State Tracking	Creates a refund case ID and stores workflow stage, next action, and audit trail.
Human Review Safety Layer	Routes generated a refund package to a human reviewer before submission to reduce hallucination or incorrect claims.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: The system uses Z.AI GLM as the central reasoning model because the hackathon requires GLM usage and because refund workflows require language understanding, classification, reasoning over incomplete information, and structured output generation.

4.3.2. Prompting Strategy: The planned prompting strategy is multi-step agentic prompting. Instead of asking GLM for a single answer, the backend prompts GLM to perform

© UMHackathon 2026

ordered reasoning steps: detect intent, classify refund reason, identify missing fields, select workflow stage, generate tasks, and return JSON. This fits the use case because refund cases require sequential decisions and not just one response.

4.3.3. Context & Input Handling:

Input Type	Handling Strategy
User message	Accepted with a minimum of 5 characters and a maximum up to approximately 6,000 characters in the prototype. Longer input is rejected or summarised before GLM call.
Order details	Optional structured fields are injected into the context window to reduce ambiguity
Evidence description	Textual evidence summary is accepted. Direct image upload is future scope
Platform policy context	Only short platform adapter rules are sent to GLM to reduce token cost.
Oversized input	The frontend warns the user; backend rejects or chunks in future versions.

4.3.4. Fallback & Failure Behavior:

- If confidence is low, the system asks clarifying questions instead of submitting a claim.
- If the platform is unknown, the system requests platform confirmation.
- If required evidence is missing, tasks are blocked until the user provides evidence.
- If GLM output is invalid JSON, the backend retries once with a stricter JSON-only prompt.
- If GLM API fails, the system returns a graceful error state and can use deterministic mock logic for demo.
- Human review is required before platform submission to reduce hallucinated or unsupported claims.

© UMHackathon 2026

5. User Stories & Use Cases

ID	User Story	Acceptance Criteria
US-01	As an online shopper, I want to describe my refund issue in my own words so that the system can understand my problem without forcing me to fill a long form	The system detects refund intent and refund reason from unstructured text.
US-02	As an online shopper, I want to know what information is missing so that I can prepare a complete refund request.	System lists missing platform, order ID, item name, amount, or evidence.
US-03	As a support agent, I want each refund request to become a structured case so that I can review it quickly	System returns case ID, status, tasks, next action, and audit trail.
US-04	As a platform operations user, I want platform-specific tasks so that refund handling can adapt to different buying platforms.	The system selects channels and tasks according to the selected platform adapter.
US-05	As a judge, I want to see GLM coordinating a workflow so that the prototype is clearly more than a chatbot.	Demo shows intent, missing-info detection, task orchestration, and structured JSON output.

- **Use Cases (Main Interactions): Refund Request Preparation Flow**

    a. The user enters a refund issue and optional order details.
    b. Frontend sends requests to the backend endpoint POST /api/refunds/analyze.
    c. Backend validates input and builds GLM context.
    d. GLM identifies intent, refund reason, missing information, workflow stage, and next action.
    e. Backend parses structured output and creates a refund case.
    f. Frontend displays tasks, missing fields, channel recommendation, and audit trail.
    g. If complete, the support agent reviews the generated refund package before platform submission.

© UMHackathon 2026
Email: umhackathon@um.edu.my

6. Features Included (Scope Definition)

MVP Feature	Description	Reason for Inclusion
Refund intake from	Frontend form accepts platform, message, order ID, item, amount, and evidence description.	Shows user-facing workflow entry point.
GLM reasoning workflow	Backend prepares context and receives structured decision output.	Core domain requirement.
Missing-information detection	System identifies missing refund details and blocks unsafe submission	Shows edge-case handling.
Platform adapter selection	Prototype supports Shopee, Lazada, TikTok Shop, Amazon, Other, and Unknown	Shows multi-platform workflow.
Case result dashboard	Frontend displays case ID, stage, tasks, next action, and raw JSON.	Useful for technical walkthrough and judging.

7. Features Not Included (Scope Control)

- No real submission to Shopee, Lazada, TikTok Shop, Amazon, or bank dispute systems in the preliminary prototype.
- No storage of real personal data, payment credentials, or private platform login sessions.
- No automatic final approval of refunds because approval depends on platform policies.
- No direct scraping of platform data or user accounts.
- No production-scale identity verification or payment dispute integration
- Image/file upload is future scope; current prototype uses text evidence descriptions.

© UMHackathon 2026

Email: umhackathon@um.edu.my

8. Assumptions & Constraints

Area	Assumption / Constraint	Design Decision
LLM Cost	An average session should use a short prompt containing only user message, structured fields, and platform adapter rules.	Use compact JSON schema and avoid sending long policy documents.
Token Limit	Prototype accepts around 6,000 characters for user messages.	Reject or summarise oversized input.
Technical	Platform APIs may not be publicly available for refund submission.	Use mock platform adapters and demonstrate workflow readiness.
Data Privacy	Refund cases may include personal/order information.	Do not store sensitive data in production; use demo data only.
Performance	GLM calls may be slower than deterministic rules.	Display loading state and cache repeated demo cases in future version.
Human Control	AI can prepare and recommend, but not guarantee approval.	Add a human review stage before submission.

9. Risks & Questions Throughout Development

Risk/ Question	Impact	Mitigation
GLM hallucination	Could generate wrong refund reason or unsupported platform instruction	Use JSON schema validation, confidence threshold, and human review.
Incomplete user input	Workflow may not be ready for submission.	Ask for missing information and block submission tasks.
Platform policy variation	Refund requirements differ by region and item type.	Use configurable platform adapters and disclose prototype limitations.
Sensitive information	Users may enter phone numbers, addresses, or payment details.	Warn users not to submit sensitive data; redact in future version
Over-reliance on automation	Users may think AI guarantees refund approval.	Display disclaimer that final decision belongs to platform.
API failure	GLM or backend may be unavailable during demo.	Provide mock mode and graceful error states.

© UMHackathon 2026

Email: umhackathon@um.edu.my